

컴프3 텀프보고서

Mini Student Shell — 학생 성적 관리 프로그램

과 목 CP3

학 번 202302529

이 름 김도연

제출일 2026. 06. 17.

1. 프로젝트 개요

이번 프로젝트는 C 언어로 학생들의 성적 정보를 관리하는 간단한 명령어 기반 셸 프로그램을 만드는 것을 목표로 했다..

학생 한 명의 정보는 학번(id), 이름(name), 점수(score) 세 가지로 구성되며, 이 데이터들은 메모리 상에서 단일 연결 리스트(singly linked list)로 관리된다. 프로그램이 종료되어도 데이터가 유지되도록 CSV 파일에 저장하고 불러오는 기능을 함께 구현했다.

프로그램은 권한에 따라 두 가지 버전으로 나뉜다. 모든 기능을 사용할 수 있는 관리자용 admin_shell과, 데이터를 조회만 할 수 있는 client_shell이다. 두 프로그램은 동일한 소스 코드를 공유하되, 컴파일 단계에서 매크로(-DADMIN_MODE, -DCLIENT_MODE)를 다르게 주어 사용할 수 있는 명령어 집합을 구분한다.

1.1 주요 기능

- 학생 정보 추가(add), 조회(find), 전체 목록 출력(list)
- 점수 수정(update), 삭제(delete), 이름·점수 기준 정렬(sort)
- 인원수·평균·최고·최저 점수 등 통계 출력(stats)
- CSV 파일로 저장(save) 및 다시 불러오기(reload)
- 표준 입력을 통한 대화형 모드와, 명령어 파일을 읽어 실행하는 배치(-f) 모드 지원
- 관리자/클라이언트 권한 분리 및 잘못된 입력에 대한 예외 처리

2. 프로그램 구조

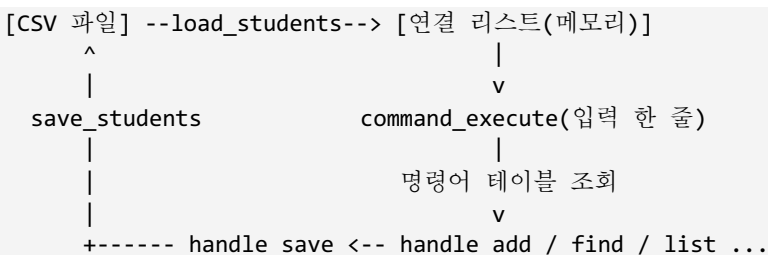
프로그램은 역할에 따라 네 개의 소스 파일로 모듈화했다. 각 모듈은 자신이 맡은 책임만 처리하도록 분리해서, 한 부분을 수정하더라도 다른 부분에 영향을 최소화하려고 했다. 모듈 간의 의존 관계는 헤더 파일(.h)을 통해 인터페이스만 공개하는 방식으로 연결된다.

2.1 모듈 구성

모듈	헤더	역할
main.c	—	프로그램 진입점. 명령행 인자 해석, 대화형/배치 셸 루프 실행
student.c	student.h	Student 구조체와 연결 리스트 조작(생성·추가·검색·삭제·정렬)
file_io.c	file_io.h	CSV 파일 읽기(load)와 쓰기(save), 형식 검증
command.c	command.h	명령어 테이블 관리, 입력 파싱, 핸들러 호출, 에러 메시지 출력

2.2 데이터 흐름

프로그램의 전체적인 흐름은 다음과 같다. 먼저 main이 CSV 파일을 읽어 연결 리스트를 메모리에 올린다. 이후 셸 루프가 사용자(또는 명령어 파일)로부터 한 줄씩 입력을 받아 command_execute로 넘기고, command_execute는 명령어 이름을 찾아 알맞은 핸들러를 호출한다. 핸들러는 student 모듈의 리스트 함수나 file_io 모듈의 파일 함수를 이용해 실제 작업을 수행한다.



이 구조에서 핵심은 command_execute가 명령어 이름과 처리 함수를 짝지은 "명령어 테이블"을 순회하면서 일치하는 핸들러를 찾아 호출한다는 점이다. 덕분에 새로운 명령어를 추가할 때 테

이블에 한 줄만 더하면 되고, 분기문(if/else)을 길게 늘릴 필요가 없다.

2.3 Admin / Client 빌드 분리

Makefile은 같은 소스 파일들을 두 번 컴파일한다. 한 번은 -DADMIN_MODE를, 다른 한 번은 -DCLIENT_MODE를 정의해서 각각 admin_shell과 client_shell 실행 파일을 만든다. 소스 코드 안에서는 #ifdef ADMIN_MODE 같은 조건부 컴파일로 모드별 차이를 구현했다.

```
all: admin client
admin:
    $(CC) $(CFLAGS) -DADMIN_MODE $(SRCS) -o admin_shell
client:
    $(CC) $(CFLAGS) -DCLIENT_MODE $(SRCS) -o client_shell
```

3. 자료구조 설계

3.1 Student 구조체

학생 한 명의 정보는 다음과 같은 Student 구조체로 표현한다. 이름은 최대 31자까지 저장할 수 있도록 32바이트 배열로 두었고, 마지막 한 칸은 항상 널 종료 문자를 위해 남겨 둔다. 자기 자신을 가리키는 포인터 next를 두어 연결 리스트의 노드 역할을 겸하도록 했다.

```
typedef struct Student {
    int id;           // 학번 (양의 정수)
    char name[32];    // 이름 (최대 31자 + 널 종료)
    int score;        // 점수 (0 ~ 100)
    struct Student *next; // 다음 노드 포인터
} Student;
```

3.2 단일 연결 리스트 선택 이유

학생 수가 미리 정해져 있지 않고 add/delete로 수시로 늘었다 줄었다 하기 때문에, 크기가 고정된 배열보다 동적으로 노드를 매달 수 있는 연결 리스트가 더 적합하다고 판단했다. 노드는 student_create에서 malloc으로 하나씩 할당하고, 더 이상 필요 없어지면 free로 즉시 반환한다.

리스트는 head 포인터 하나로 관리되며, list_append는 끝까지 따라가 새 노드를 꼬리에 붙인다. 따라서 입력한 순서대로 데이터가 유지된다. 삽입 위치를 끝으로 정한 것은 CSV에서 읽은 순서와 화면에 보이는 순서를 일치시키기 위해서다.

3.3 리스트 조작 함수

함수	시간복잡도	설명
list_append	$O(n)$	리스트 맨 뒤에 노드 추가
list_find	$O(n)$	id가 일치하는 노드를 앞에서부터 탐색
list_delete	$O(n)$	id로 노드를 찾아 연결을 잇고 free
list_count	$O(n)$	노드 개수 세기
list_free	$O(n)$	모든 노드를 순회하며 메모리 해제
list_sort_by_*	$O(n^2)$	버블 정렬로 이름/점수 기준 정렬

3.4 메모리 관리 방식

메모리 누수를 막기 위해 할당과 해제의 책임을 분명히 했다. 노드 생성은 오직 `student_create`에서만 `malloc`으로 이루어지고, 해제는 `list_delete`(개별 노드)와 `list_free`(전체)에서만 일어난다. 프로그램이 종료되거나 reload로 데이터를 다시 읽을 때는 반드시 기존 리스트를 `list_free`로 비운 뒤 새로 채우기 때문에, 같은 메모리를 두 번 해제하거나 해제하지 않고 잃어버리는 일이 없도록 했다.

정렬은 노드의 연결(next 포인터)을 바꾸는 대신, 두 노드의 데이터(id, name, score)만 맞바꾸는 방식으로 구현했다. 포인터를 직접 재배치하는 것보다 구현이 단순하고 실수할 여지가 적기 때문이다.

4. CSV 파일 형식

데이터는 사람이 눈으로 확인하고 수정하기 쉬운 CSV(Comma-Separated Values) 형식으로 저장한다. 파일의 첫 줄은 반드시 헤더 "id,name,score"여야 하며, 그 아래로 학생 한 명당 한 줄씩 쉼표로 구분된 세 개의 값이 들어간다.

4.1 파일 예시 (students.csv)

```
id,name,score
1,Alice,90
2,Bob,85
```

```
3,Caleb,95
4,Kim,83
5,Lee,90
6,Park,97
```

4.2 불러오기(load) 과정

load_students는 파일을 열어 첫 줄이 정확히 "id,name,score"인지 먼저 확인한다. 헤더가 다른 형식이 잘못된 파일로 보고 읽기를 중단한다. 헤더를 통과하면 한 줄씩 읽으면서 쉼표 두 개를 기준으로 세 토막으로 나눈다. 각 줄마다 id가 양의 정수인지, score가 0~100 범위인지, 이름이 비어 있지 않은지를 검사하고, 문제가 있는 줄은 경고 메시지를 출력한 뒤 건너뛰고 정상적인 줄만 리스트에 추가한다.

이렇게 한 줄에 오류가 있어도 전체를 포기하지 않고 나머지 정상 데이터는 살리는 방식으로 만들어서, 파일 일부가 손상돼도 프로그램이 최대한 동작하도록 했다.

4.3 저장(save) 과정

save_students는 파일을 쓰기 모드로 열고 가장 먼저 헤더 줄을 출력한 다음, 연결 리스트를 처음부터 끝까지 순회하며 각 노드를 "id,name,score" 형식의 한 줄로 기록한다. 저장이 끝나면 기록한 학생 수를 반환한다. 저장은 항상 파일 전체를 새로 쓰는 방식이라, 메모리 상의 리스트 상태가 그대로 파일에 반영된다.

5. 명령어 명세

프로그램이 지원하는 명령어와 사용법은 다음과 같다. 권한 열의 "공통"은 admin과 client 양쪽에서 모두 쓸 수 있는 명령어이고, "관리자"는 admin_shell에서만 사용할 수 있는 명령어다. 클라이언트에서 관리자 전용 명령어를 입력하면 권한 거부 메시지가 출력된다.

명령어	사용법	권한	설명
find	find <id>	공통	학번으로 학생 한 명 조회
list	list	공통	전체 학생 목록 출력
stats	stats	공통	인원수·평균·최고·최저 점수

명령어	사용법	권한	설명
reload	reload	공통	CSV에서 데이터 다시 읽기
help	help	공통	사용 가능한 명령어 목록 표시
clear	clear	공통	화면 지우기
exit	exit	공통	프로그램 종료
add	add <id> <name> <score>	관리자	학생 추가
update	update <id> <score>	관리자	학생 점수 수정
delete	delete <id>	관리자	학번으로 학생 삭제
sort	sort <name score>	관리자	이름/점수 기준 정렬
save	save	관리자	현재 데이터를 CSV에 저장

5.1 인자 규칙과 예외 상황

- id 는 1 이상의 정수여야 한다. 0 이하이거나 숫자가 아니면 invalid argument 오류가 난다.
- score 는 0 이상 100 이하의 정수여야 한다. 범위를 벗어나면 invalid score 오류가 난다.
- add 시 이미 존재하는 id 를 넣으면 duplicate ID 오류로 거부된다.
- find, update, delete 에서 없는 id 를 지정하면 student not found 오류가 난다.
- 인자 개수가 부족하면 missing argument, 정의되지 않은 명령어는 Unknown command or permission denied 메시지가 출력된다.

6. Admin / Client Program 설계

같은 프로그램을 권한에 따라 둘로 나눈 핵심 아이디어는, 모드별로 "명령어 테이블"의 내용을 다르게 컴파일하는 것이다. command.c 안에 명령어 테이블이 두 벌 정의되어 있고, #ifdef로 둘 중 하나만 컴파일에 포함된다.

6.1 명령어 테이블 (조건부 컴파일)

```
#ifdef ADMIN_MODE
```

```
static const Command commands[] = {
    {"save",  handle_save,  ...},
    {"add",   handle_add,   ...},
    {"delete", handle_delete, ...},
    {"update", handle_update, ...},
    {"sort",  handle_sort,  ...},
    {"find",  ...}, {"list", ...}, {"stats", ...}, ...
};
#elif defined(CLIENT_MODE)
static const Command commands[] = {
    {"find", ...}, {"list", ...}, {"stats", ...},
    {"reload", ...}, {"help", ...}, {"exit", ...}
};
#endif
```

client_shell의 테이블에는 add, delete, update, save, sort 같은 데이터를 바꾸는 명령어가 처음부터 들어 있지 않다. 따라서 클라이언트에서 add를 입력하면 테이블에서 일치하는 명령어를 찾지 못하고, command_execute가 마지막에 "Unknown command or permission denied" 메시지를 출력한다. 즉 권한 제어가 단순한 조건문이 아니라 컴파일 시점에 명령어 자체를 제거하는 방식으로 이루어져, 클라이언트 실행 파일에는 쓰기 기능 코드 경로가 호출될 여지가 없다.

이런 설계는 모드별로 따로 소스를 관리하지 않아도 되고(코드 중복이 없음), 클라이언트 바이너리에서 쓰기 명령이 실수로 노출될 위험도 없다는 장점이 있다.

6.2 변경 사항 추적 (dirty 플래그)

admin 모드에서는 add·update·delete·sort로 데이터가 바뀌면 g_dirty 플래그를 1로 표시하고, save하면 0으로 되돌린다. 저장하지 않은 변경이 있는 상태에서 exit를 입력하면 "Warning: you have unsaved changes." 경고를 띄워, 작업 내용을 실수로 날리지 않도록 했다. 이 기능은 client 모드에는 데이터 변경이 없으므로 포함되지 않는다.

6.3 실행 방법

```
$ make # admin_shell, client_shell 둘 다 빌드
$ ./admin_shell students.csv # 관리자 모드
$ ./client_shell students.csv # 클라이언트 모드
$ ./admin_shell -f commands.txt students.csv # 배치 모드
```

7. 예외 처리 설계

프로그램이 잘못된 입력 때문에 비정상 종료되지 않도록, 오류 상황을 ShellResult라는 enum으

로 분류했다. 각 핸들러는 작업이 실패하면 그 원인에 해당하는 결과값을 돌려주고, `command_execute`가 이를 받아 사람이 읽을 수 있는 메시지로 바꿔 출력한다. 오류가 나도 프로그램은 멈추지 않고 다음 입력을 계속 받는다.

7.1 오류 분류와 메시지

ShellResult 값	출력 메시지
SHELL_ERR_UNKNOWN_COMMAND	Unknown command or permission denied.
SHELL_ERR_INVALID_ARGUMENT	Error: invalid argument.
SHELL_ERR_MISSING_ARGUMENT	Error: missing argument.
SHELL_ERR_FILE_OPEN	Error: cannot open file.
SHELL_ERR_FILE_WRITE	Error: cannot write file.
SHELL_ERR_STUDENT_NOT_FOUND	Error: student not found.
SHELL_ERR_DUPLICATE_STUDENT	Error: duplicate ID.
SHELL_ERR_INVALID_SCORE	Error: invalid score.

7.2 처리하는 예외 상황

- 입력 검증: id 가 정수가 아니거나 0 이하인 경우, score 가 0~100 을 벗어난 경우, 이름에 심표가 들어간 경우(CSV 형식이 깨지므로 금지)
- 데이터 무결성: 중복 학번 추가 시도, 존재하지 않는 학번 조회·수정·삭제 시도
- 파일 오류: 파일이 없거나 열 수 없는 경우, CSV 헤더가 잘못된 경우, 손상된 줄이 섞인 경우(해당 줄만 건너뛰)
- 입력 안전성: fgets 로 한 줄을 고정 버퍼(256 바이트)에 읽어 버퍼 오버플로우를 방지하고, strncpy 로 이름을 복사할 때도 항상 널 종료를 보장

7.3 배치 모드의 줄 번호 안내

명령어 파일(-f)을 실행할 때 어떤 줄에서 오류가 났는지 알 수 있도록, 배치 모드에서는 오류 메시지 뒤에 "Skipped line N."을 덧붙인다. 예를 들어 명령어 파일의 두 번째 명령에서 중복 학번

이 발생하면 "Error: duplicate ID. Skipped line 2."처럼 출력되어, 어느 명령이 무시됐는지 바로 확인할 수 있다.

8. 테스트 케이스와 결과

아래는 실제로 프로그램을 컴파일해 실행한 결과다. 입력한 명령어와 그에 대한 프로그램의 출력을 그대로 옮겨 적었다.

8.1 빌드

```
$ make
gcc -Wall -Wextra -std=c11 -DADMIN_MODE main.c student.c file_io.c command.c -o admin_shell
gcc -Wall -Wextra -std=c11 -DCLIENT_MODE main.c student.c file_io.c command.c -o client_shell
(경고 0개로 정상 빌드)
```

8.2 조회 기능 (list / find / stats)

```
$ ./admin_shell students.csv
[Admin Program]
Loaded 6 students from students.csv.
admin> list
ID      Name    Score
1       Alice   90
2       Bob     85
3       Caleb   95
4       Kim     83
5       Lee     90
6       Park    97
admin> find 3
ID: 3
Name: Caleb
Score: 95
admin> stats
Count: 6
Average: 90.0
Max: 97
Min: 83
```

8.3 추가 / 수정 / 삭제 / 정렬 / 저장

```
admin> add 7 Choi 78
Student added.
admin> update 2 99
Student updated.
admin> delete 5
Student deleted.
admin> sort score
Students sorted by score.
admin> list
```

ID	Name	Score
7	Choi	78
4	Kim	83
1	Alice	90
3	Caleb	95
6	Park	97
2	Bob	99

```
admin> save
Saved 6 students to students.csv.
```

정렬 결과가 점수 오름차순(78→83→90→95→97→99)으로 잘 정렬되었고, save 후 파일을 열어 보면 메모리의 리스트 순서와 동일하게 저장된 것을 확인할 수 있었다.

8.4 클라이언트 권한 제어

```
$ ./client_shell students.csv
[Client Program]
Loaded 6 students from students.csv.
client> list
ID      Name    Score
1       Alice   90
... (생략)
client> add 9 Test 50
Unknown command or permission denied.
client> delete 1
Unknown command or permission denied.
client> find 2
ID: 2
Name: Bob
Score: 85
```

클라이언트에서는 list, find 같은 조회 명령은 정상 동작하지만 add, delete 등 데이터를 바꾸는 명령은 모두 거부되는 것을 확인했다.

8.5 예외 처리 테스트

```
admin> add 1 Alice 90      # 이미 있는 학번
Error: duplicate ID.
admin> add 8 Bad 150       # 점수 범위 초과
Error: invalid score.
admin> update 999 80      # 없는 학번 수정
Error: student not found.
admin> delete 999         # 없는 학번 삭제
Error: student not found.
admin> find abc           # 숫자가 아닌 id
Error: invalid argument.
admin> add 9              # 인자 부족
Error: missing argument.
admin> foobar             # 정의되지 않은 명령
Unknown command or permission denied.
```

8.6 배치 모드 (-f commands.txt)

```
$ ./admin_shell -f commands.txt students.csv
[Admin Program]
Loaded 6 students from students.csv.
[command file:1] list
ID      Name    Score
1       Alice   90
...
[command file:2] add 4 David 88
Error: duplicate ID. Skipped line 2.
[command file:3] update 99 70
Error: student not found. Skipped line 3.
[command file:4] find 4
ID: 4
Name: Kim
Score: 83
[command file:5] save
Saved 6 students to students.csv.
[command file:6] exit
Goodbye.
```

배치 모드에서는 각 명령 앞에 [command file:N] 표시가 붙어 어떤 명령이 실행 중인지 보여 주고, 오류가 난 줄은 "Skipped line N."으로 안내한 뒤 다음 명령을 계속 처리하는 것을 확인했다.

8.7 손상된 CSV 불러오기

점수가 범위를 벗어난 줄과 심표 개수가 맞지 않는 줄을 일부러 섞은 파일을 불러오게 했더니, 문제가 있는 줄만 경고와 함께 건너뛰고 정상 데이터 2명만 정상적으로 읽어 들었다.

```
Error: invalid score at line 3.
Error: invalid CSV format at line 4.
Loaded 2 students from mal.csv.
```

8.8 테스트 결과 요약

테스트 항목	결과	비고
빌드 (경고 포함)	통과	-Wall -Wextra 경고 0개
조회(list/find/stats)	통과	값 정상 출력
추가/수정/삭제/정렬	통과	정렬·저장 순서 일치
클라이언트 권한 제어	통과	쓰기 명령 모두 거부
예외 처리 8종	통과	비정상 종료 없음
배치 모드	통과	줄 번호 안내 정상

테스트 항목	결과	비고
손상 CSV 처리	통과	오류 줄만 건너뛴

9. 한계점과 개선 방향

9.1 한계점

- 탐색·삭제가 모두 연결 리스트를 앞에서부터 훑는 $O(n)$ 방식이라, 학생 수가 아주 많아지면 느려진다. 학번을 키로 하는 해시 테이블이나 이진 탐색 트리를 쓰면 평균 탐색 속도를 크게 줄일 수 있을거 같다.
- 정렬이 버블 정렬($O(n^2)$)이고, 노드 연결을 바꾸지 않고 데이터만 교환하는 방식이라 데이터가 많아지면 비효율적이다. 병합 정렬 같은 $O(n \log n)$ 알고리즘으로 바꾸는 것이 나아보인다.
- 이름이 31 자로 제한되어 있고, 이름에 쉼표나 공백을 자유롭게 쓸 수 없다.
- 권한이 컴파일 시점에 고정되어, 실행 중에 로그인이나 비밀번호로 권한을 바꾸는 기능은 없다.

9.2 개선 방향

- 자료구조를 해시 테이블로 교체해 find/delete 를 평균 $O(1)$ 에 가깝게 개선
- 정렬 알고리즘을 병합 정렬로 교체하고, 정렬 기준(오름/내림차순)을 인자로 선택 가능하게 확장
- 로그인 기반 권한 관리와 변경 이력(로그) 기록 기능 추가
- CSV 뿐 아니라 JSON 등 다른 형식의 입출력 지원, 그리고 단위 테스트 자동화 도입

감사합니다.